Combined-Policy Imitation Learning on Specialized GA Bots in X-Pilot

Muhammad Abdullah COM 407: Computational Intelligence Fall 2025

Abstract

This project investigates whether combining multiple specialist policies can produce a robust, end-to-end imitation agent for X-Pilot. I evolved three Genetic Algorithm (GA) specialists **Thruster**, **Shooter**, and **Navigator** collected millions of structured state-action demonstrations, trained per-expert neural imitation models, and built a gating network to blend the experts into a single combined policy. The combined model produces smoother behavior than each neural expert and outperforms the neural specialists, while the original GA agents remain strong baselines.

Motivation

Previous imitation work for X-Pilot in the course controlled only a subset of actions and did not produce reliable end-to-end controllers. There was a clear gap: no single imitation model convincingly replaced a rule-based or GA agent, and nobody had explored whether *specialized* experts each optimized for a different competency could be combined to produce a better overall policy. This project addresses that gap by (1) building strong specialist GAs, (2) training neural imitation models from their demonstrations, and (3) combining those models with a learned gating mechanism.

Goals

- Produce full end-to-end imitation agents that control all four discrete actions: thrust, turnLeft, turnRight, shoot.
- Train three specialist GAs (Thruster, Shooter, Navigator) and collect demonstration data from competitive play.
- Train neural behavior-cloning models on each dataset.
- Learn a gating network that blends expert outputs into a single combined policy and evaluate whether it improves performance and robustness.

GA development & the Survivor GA

Before evolving specialist agents, I trained a Survivor GA to optimize longevity in the arena. Each chromosome encoded 11 genes (55 bits total) that parameterize threshold values for a rule-based controller (e.g., front-distance thresholds, aim tolerance). Populations were scaled so gene values produced realistic ranges (for example, front-distance ×20). I trained the Survivor GA for 150 generations in an empty arena; many parameters converged by generation ~120 and the best survivors rarely crashed.

Why the Survivor GA mattered
Seeding specialist populations with a small number of proven survivor chromosomes had three practical benefits:

1. **Faster convergence** specialists inherited survival heuristics rather than relearning them from scratch.

2. **Reduced degenerate runs** fewer early wall-crash episodes during specialist evolution.

3. **Clear separation of concerns** specialists could focus on their objective (thrusting, shooting, or exploration) while the seeded survivors maintained baseline survivability.

Specialized GA experts

All specialist GAs used populations of 128 chromosomes (118 random + 10 seeded survivors) and were trained against a rule-based Expert agent.

Thruster (fitness: thrust usage)
Trained 150 generations. Learned strategic thrusting behavior and excellent wall avoidance. The Thruster often turns near walls to align before thrusting; it uses thrust sparingly for speed and escape maneuvers. Unexpectedly, it also developed decent shooting and dodging behaviors even without a score objective.

Shooter (fitness: score)
Trained 150 generations. Focused on accuracy and scoring. Learned to move slowly for better aim, exploit aim mechanics when stationary, and keep strong defensive positioning. This agent frequently dominated the expert rule-based baseline.

Navigator (fitness: distance traveled)
Trained 120 generations. Became the fastest and most exploratory agent, traveling across the map efficiently with strong wall avoidance. It is effective at shooting while moving but is slightly more prone to high-speed crashes in complex situations.

All three GAs reliably escaped corner traps and demonstrated distinct, specialist behavior patterns that are useful for mixing later.

Data collection

Each GA was run in competitive matches against the expert agent while logging per-tick state variables into CSV files.

- Raw logs: 21 variables per tick.
- Training inputs: 17 normalized features (range 0–1).
- Outputs: 4 discrete action labels, thrust, shoot, turnLeft, turnRight.
- Dataset size: roughly **5.0 million** rows for Shooter, **4.8 million** for Navigator, and **4.8 million** for Thruster (≈15M total). These large demonstration sets ensure high-quality coverage across many game situations.

Neural imitation models

For each specialist, I trained a supervised behavior-cloning model that maps 17 inputs → 4 action probabilities.

- Architectures:
    - **Shooter & Thruster:** 4 hidden layers, 256 → 128 → 64 → 32 (fully connected).
    - **Navigator:** 4 hidden layers, 196 → 128 → 64 → 32 (first layer reduced to lighten computation on the largest dataset).
- Optimizer: **Adam**
- Learning rate: **0.001**
- Train/test split: **80% / 20%**
- Loss/validation tracked, with early stopping applied where appropriate.

Model behavior, qualitative summary

Shooter model: Excellent wall avoidance and precise targeting; tends to shoot abruptly and sometimes when unnecessary.
Thruster model: Replicates GA thrust usage well, smooth avoidance, but occasionally crashes in complex situations.
Navigator model: Superb at traversing the map and intermittent high-speed navigation; sometimes unstable at extreme speeds.

Quantitative results (test set)

Per-action accuracies measured on held-out data:

Navigator

- Thrust: 95.90%
- TurnLeft: 98.04%
- TurnRight: 97.93%
- Shoot: 98.99%
- Exact match (all 4 actions): **92.52%**

Shooter

- Thrust: 96.52%
- TurnLeft: 97.99%
- TurnRight: 97.91%
- Shoot: 99.17%
- Exact match: **93.39%**

Thruster

- Thrust: 94.50%
- TurnLeft: 97.84%

- TurnRight: 97.77%
- Shoot: 99.14%
- Exact match: **91.08%**

These high per-action accuracies indicate the networks learned to imitate the specialists closely; exact-match rates show how often the full action vector matches the expert on held-out ticks.

Combined policy: mixture of experts

To leverage specialist strengths, I trained a small **gating network** that maps the same 17-dim input to a softmax over experts: $[w_1, w_2, w_3]$. Each expert network outputs an action probability vector a_i (for [thrust, turnLeft, turnRight, shoot]). The final action is a weighted combination:

$$a = w_1 \cdot a_1 + w_2 \cdot a_2 + w_3 \cdot a_3$$

This produces a smooth, continuous action distribution that is discretized to controls (e.g., via argmax or thresholding) for deployment.

Behavioral effect: the combined policy inherits the navigation and wall-avoidance strengths of Navigator/Thruster and the precision of Shooter. It behaves much smoother than individual neural experts (fewer abrupt shots, better obstacle avoidance), and in neural-vs-neural matches the combined model consistently outperforms the separate neural experts.

Limitations: In direct GA-vs-GA comparisons, the original GAs sometimes still outperform the combined neural policy; the combined policy can also occasionally over-emphasize Navigator in open spaces and adopt risky high-speed movement.

Conclusions & next steps

- Combining specialized experts via a learned gating network yields a smoother, more versatile imitation agent that outperforms the separate neural experts.
- GA agents remain strong baselines, to close the gap, fine-tuning the combined policy with reinforcement learning or distilling the mixture into a compact single model are promising next steps.
- Further work: explore richer gating architectures (contextual or hierarchical), test robustness across multiple maps and opponent behaviors, and investigate policies that optimize long-term objectives (survival + score).

Appendix: quick facts

- Map: lifeless.xp
- Data: Shooter 5.0M rows, Navigator 4.8M, Thruster 4.8M (~15M total)
- Inputs: 17 normalized features (see list above)
- Outputs: 4 discrete actions: thrust, turnLeft, turnRight, shoot
- Models: 4 hidden layers (first layer 256 for Shooter/Thruster; 196 for Navigator as trained)
- Optimizer: Adam, LR = 0.001, train/test = 80/20